

# BorrowHub

## Smart Community Rental Platform

### PROJECT REPORT

Theme: Community-Based Rental Platform

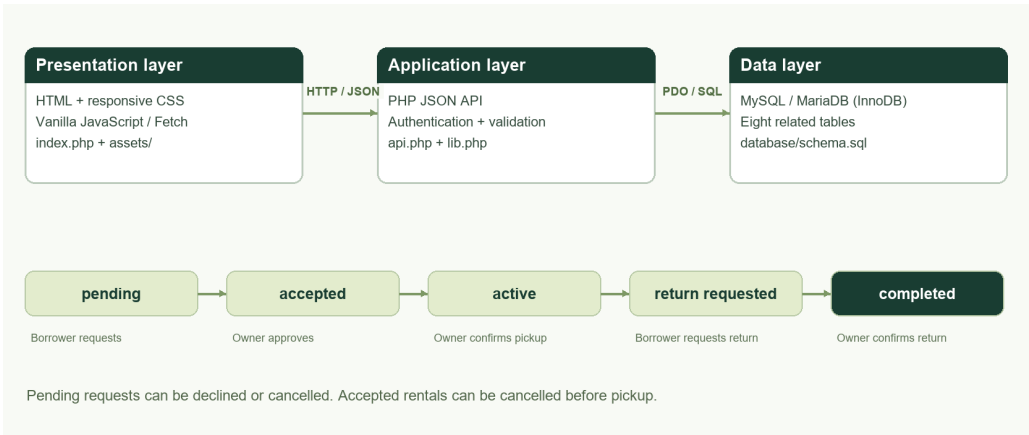


Figure 1. BorrowHub connects community members through a web application and a relational database.

| Group member    | Index number |
|-----------------|--------------|
| K. M. Aashik    | ITT/2024/001 |
| M. R. M. Rislam | ITT/2024/090 |

Date: 25 September 2026

Local implementation: PHP 8.2 • MySQL-compatible MariaDB • XAMPP

*Borrow what you need. Rent what you do not use.*

# Report contents

| Section                     | Coverage                                                                             |
|-----------------------------|--------------------------------------------------------------------------------------|
| 1. Cover page               | Project identity, theme, members, index numbers and date.                            |
| 2. Project overview         | Purpose, audience, objectives, features and scope.                                   |
| 3. System design            | Architecture, database relationships, schema, folder structure and API organisation. |
| 4. Implementation           | Major features, application screenshots, workflow, security and validation.          |
| 5. Challenges and solutions | Design problems, observed development issues and their resolutions.                  |
| 6. Individual contribution  | Joint participation of both members across the project.                              |
| 7. Setup instructions       | XAMPP installation, SQL import, configuration, startup and troubleshooting.          |
| 8. References               | Project evidence and official technical documentation.                               |

## Executive summary

BorrowHub is a community rental application that connects people who own underused items with people who need them temporarily. A member can both list items and request rentals. Owners review requests, confirm pickup and confirm returned items. The application preserves rental history, supports private conversations between rental participants, and allows a borrower to review a completed rental.

The implementation uses a responsive browser interface, a PHP JSON API and an eight-table relational database. It includes member and administrator workflows and is designed to run locally through XAMPP. Payments and refundable deposits are arranged directly between members; online payment processing is outside the implemented scope.

## Evidence and report basis

This report is based on the supplied project proposal [1], the installed BorrowHub source and schema [2], and the recorded verification results [3]. Screenshots were captured on 25 September 2026 from the implemented application running against a separate demonstration database. Demo people, messages, reviews and rentals are illustrative and do not represent real transactions.

Implementation details are reported as they exist. Although Bootstrap 5 was proposed, the delivered interface uses custom responsive CSS. The team confirmed that all project activities were carried out together; the contribution section records shared participation rather than inventing separate task ownership.

## 2. Project overview

### 2.1 Purpose and problem statement

Items such as cameras, calculators, laptops, drills and camping equipment are useful but may be needed only occasionally. Purchasing them for a short activity can be expensive, while equipment already owned by neighbours may sit unused. BorrowHub addresses this problem by providing a structured way to discover local items, agree on rental dates and manage the return process [1].

### 2.2 Target audience

| Audience                               | Typical need                                                     |
|----------------------------------------|------------------------------------------------------------------|
| University students                    | Short-term access to calculators, laptops and study equipment.   |
| Professionals and hobbyists            | Cameras, electronic equipment and tools for a specific activity. |
| Local households and community members | Equipment for home tasks, outdoor trips or events.               |
| Item owners                            | A way to list underused possessions and agree on paid rentals.   |

### 2.3 Objectives and key features

- Provide a responsive interface for searching by keyword, category, town, daily price and availability.
- Support registration, login, private account details and secure password handling.
- Let members add photos, set daily prices and deposits, edit listings, or pause availability.
- Manage requests, approvals, pickup, returns, cancellations and rental history.
- Support rental-specific messages and reviews after completed rentals.
- Provide administration for listings, members, categories and support messages.

### 2.4 Implemented scope

Prices are displayed in Sri Lankan rupees and the application time zone is Asia/Colombo. Rental periods are 1–90 days, beginning today or within the next year. Reminders appear in the dashboard; messages use manual refresh. There is no payment gateway, escrow, insurance service, live chat socket, email delivery or SMS notification service in this version.

## 3. System design

### 3.1 Architecture and request flow

BorrowHub separates presentation, application logic and persistent storage. index.php serves the page shell; assets/app.js renders views using hash-based navigation. The browser sends JSON requests to api.php using Fetch [4]. The PHP layer validates input and permissions before reading or updating MySQL through PDO prepared statements [5].

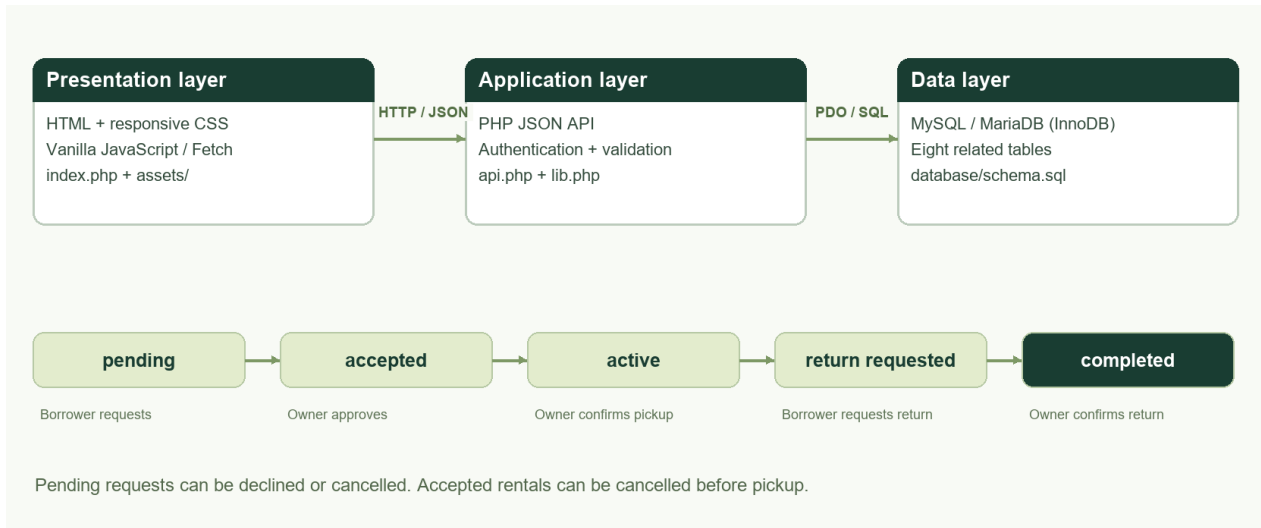


Figure 2. Three-layer architecture and the normal rental lifecycle.

| Layer        | Implementation                | Responsibility                                                            |
|--------------|-------------------------------|---------------------------------------------------------------------------|
| Presentation | HTML, CSS, vanilla JavaScript | Forms, responsive layouts, navigation, visual feedback and cost previews. |
| Application  | PHP: api.php and lib.php      | Sessions, CSRF validation, business rules, uploads and authorization.     |
| Persistence  | MySQL / MariaDB, InnoDB       | Users, listings, rentals, reviews, messages and administrative records.   |

### 3.2 Roles and permissions

Visitors can browse public items, read details and submit a support message. A signed-in member can both borrow and lend; there is no separate permanent “owner” account type. Only the relevant owner may edit a listing or approve its request. Only a rental’s borrower and owner may read its messages. Administrators can moderate listings and members and view rental summaries, but the administration overview does not expose private message bodies.

### 3.3 Rental state model

The principal sequence is pending → accepted → active → return\_requested → completed. Pending requests can be rejected or cancelled. Accepted rentals can be cancelled before pickup. Completion requires the borrower’s return request and the owner’s confirmation. These transitions are checked on the server, rather than trusting which buttons the browser displays.

## 3.4 Database relationships

The database contains eight InnoDB tables using utf8mb4 text encoding. Primary keys identify records, foreign keys retain relationships, and unique constraints prevent duplicate emails, category names, reset-token hashes and reviews for the same rental. Figure 3 summarises the model; the following pages describe the stored fields.

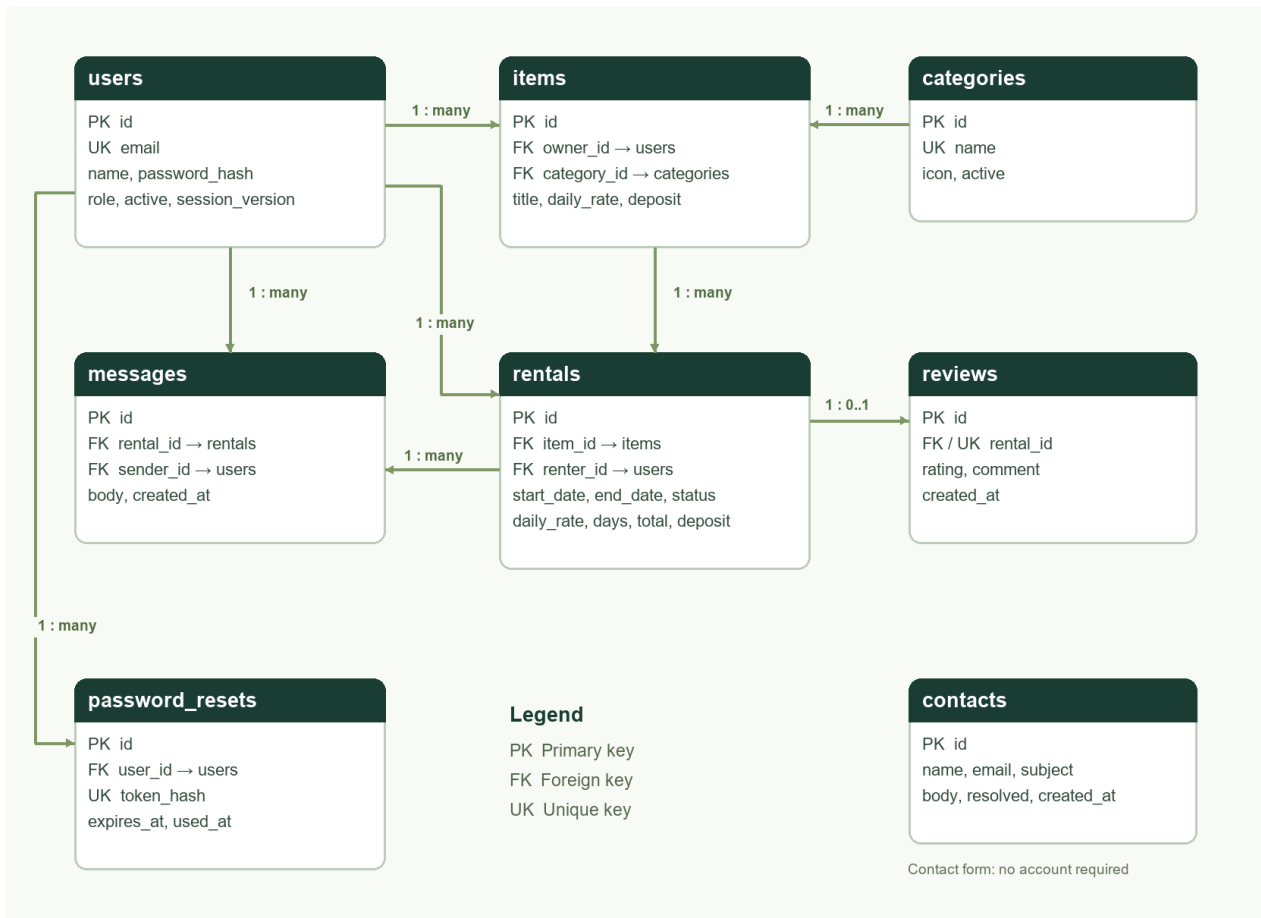


Figure 3. Entity relationships. The contact form is independent of a registered account.

| Relationship               | Meaning                                                                  |
|----------------------------|--------------------------------------------------------------------------|
| users → items / rentals    | A user owns many listings and may borrow through many rental records.    |
| categories → items         | Each listing belongs to one category; a category contains many listings. |
| items → rentals            | An item can have many requests and historical rentals.                   |
| rentals → reviews          | A rental has zero or one review, enforced by a unique rental_id.         |
| rentals / users → messages | Each message belongs to one rental and one sender.                       |
| users → password_resets    | A member can have multiple issued reset-token records.                   |

## 3.5 Database schema: core entities

Source of truth: database/schema.sql [2]. PK means primary key; FK means foreign key; UK means unique key. IDs are unsigned, auto-incrementing integers unless noted. Optional fields are identified below.

| Table      | Fields and SQL types                                                                                             | Purpose / constraints                                                                                         |
|------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| users      | id BIGINT (PK); name VARCHAR(100); email VARCHAR(190) (UK)                                                       | Member identity; email uniquely identifies the login account.                                                 |
| users      | password_hash VARCHAR(255); city VARCHAR(100); phone VARCHAR(30)                                                 | Stores a password hash, town and optional phone text (empty by default).                                      |
| users      | role ENUM(member, admin); active TINYINT(1); session_version INT; created_at TIMESTAMP                           | Role controls administration. Account status and session version support suspension and session invalidation. |
| categories | id INT (PK); name VARCHAR(80) (UK); icon VARCHAR(30); active TINYINT(1)                                          | Groups items and allows a category to be enabled or disabled.                                                 |
| items      | id BIGINT (PK); owner_id BIGINT (FK); category_id INT (FK)                                                       | Links each listing to its owner and category.                                                                 |
| items      | title VARCHAR(120); description TEXT; city VARCHAR(100); item_condition ENUM(Like new, Good, Fair)               | Describes the item, locality and condition.                                                                   |
| items      | daily_rate DECIMAL(10,2); deposit DECIMAL(10,2)                                                                  | Current listing price and refundable deposit; both validated by PHP.                                          |
| items      | image VARCHAR(150) NULL; available TINYINT(1); hidden TINYINT(1); deleted_at DATETIME NULL; created_at TIMESTAMP | Image reference, owner availability, admin moderation and soft-deletion timestamp.                            |
| rentals    | id BIGINT (PK); item_id BIGINT (FK); renter_id BIGINT (FK)                                                       | A request connecting a borrower with an item.                                                                 |
| rentals    | start_date DATE; end_date DATE; days INT; daily_rate DECIMAL(10,2); total DECIMAL(12,2); deposit DECIMAL(10,2)   | Agreed dates and a price snapshot; changes to a listing do not rewrite existing rental prices.                |
| rentals    | status ENUM; note VARCHAR(1000); created_at TIMESTAMP; updated_at TIMESTAMP                                      | Status is one of pending, accepted, active, return_requested, completed, rejected or cancelled.               |

### Indexes

The items table indexes browsing flags/category and owner\_id. The rentals table indexes item\_id, status and date boundaries for availability checks, plus renter\_id for member history. These support the main browsing and booking operations without changing the business rules.

## 3.6 Database schema: supporting entities

| Table           | Fields and SQL types                                                                                                              | Purpose / constraints                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| reviews         | id BIGINT (PK); rental_id BIGINT (FK, UK); rating TINYINT; comment VARCHAR(1000); created_at TIMESTAMP                            | One borrower review for each completed rental. PHP requires a rating from 1 to 5.        |
| messages        | id BIGINT (PK); rental_id BIGINT (FK); sender_id BIGINT (FK); body VARCHAR(2000); created_at TIMESTAMP                            | Private rental conversation. Indexed by rental_id and id for ordered retrieval.          |
| contacts        | id BIGINT (PK); name VARCHAR(100); email VARCHAR(190); subject VARCHAR(150); body TEXT; resolved TINYINT(1); created_at TIMESTAMP | Support messages can be submitted without an account. Administrators mark them resolved. |
| password_resets | id BIGINT (PK); user_id BIGINT (FK); token_hash CHAR(64) (UK); expires_at DATETIME; used_at DATETIME NULL                         | Stores hashed, expiring recovery tokens. A used token cannot be redeemed again.          |

## 3.7 Integrity and business rules

- A member cannot request a rental for an item they own.
- A pending request does not reserve dates. An accepted, active or return-requested rental blocks overlapping bookings.
- Approval runs inside a transaction and locks the item row before checking conflicting dates. This serialises approval decisions for that item [6].
- The server derives the price from the listing and rental duration; it does not accept a client-supplied total.
- A listing with confirmed or active rentals cannot be removed. Soft deletion retains historical records and rejects pending requests.
- A review is accepted only from the rental's borrower after completion. A unique rental\_id provides a second line of protection against duplicates.
- Password resets expire after 30 minutes and invalidate earlier sessions when redeemed.

## Normalization and deliberate snapshots

Users, categories, items, rentals and messages are stored as separate related records to avoid repeating account and category details. Rental rate, total and deposit are deliberately copied into the rental record because they describe the agreement at booking time. This historical snapshot is different from accidentally duplicating the current listing price.

## 3.8 Folder structure and API organisation

|                         |                                       |
|-------------------------|---------------------------------------|
| borrowhub/              |                                       |
| — index.php             | Page shell and security headers       |
| — api.php               | JSON endpoints and workflow rules     |
| — lib.php               | PDO, validation, sessions and uploads |
| — config.php            | Database connection settings          |
| — setup.php             | CLI schema, categories and demo setup |
| — reset-password.php    | CLI reset-token generator             |
| — install.ps1           | Interactive XAMPP setup               |
| — router.php            | Optional development-server router    |
| — .htaccess             | Apache access restrictions            |
| — assets/               |                                       |
| — app.js                | Views, routing and form actions       |
| — art.js                | SVG illustrations and icons           |
| — style.css             | Responsive presentation               |
| — favicon.svg           | Browser icon                          |
| — database/             |                                       |
| — schema.sql            | Eight-table database schema           |
| — ERD.md                | Relationship documentation            |
| — uploads/              | Validated item photographs            |
| — tests/integration.cjs | HTTP/MySQL integration checks         |
| — README.md             | Setup and operating guide             |
| — VERIFICATION.md       | Recorded verification evidence        |

The application is installed independently at C:\xampp\htdocs\borrowhub and uses the separate borrowhub database. Pages are selected with URL fragments such as #browse, #item?id=1 and #dashboard; there is no separate PHP file for every screen.

| API action group        | Representative actions                                       | HTTP method                    |
|-------------------------|--------------------------------------------------------------|--------------------------------|
| Accounts                | session; register, login, logout, profile, password, reset   | GET session; POST mutations    |
| Discovery               | categories, items, item                                      | GET                            |
| Listings                | item_save, item_delete                                       | POST                           |
| Rental workflow         | rentals; book, rental_status, review                         | GET history; POST changes      |
| Conversations / support | messages; message_send, contact                              | GET messages; POST submissions |
| Administration          | admin; admin_item, admin_user, admin_contact, admin_category | GET summary; POST changes      |

Requests use api.php?action=... . Most mutations accept JSON; listing forms use multipart/form-data for photos. All write operations require the session CSRF token, and PHP enforces the necessary role or ownership.

## 4. Implementation

### 4.1 Homepage and item discovery

The homepage introduces the sharing concept, provides search shortcuts and links to the six starting categories. The browse view retrieves stored listings and supports keyword, category, city, maximum price, availability and sorting filters. Results are paginated in groups of twelve. Public browsing excludes hidden or deleted listings and inactive owners/categories.

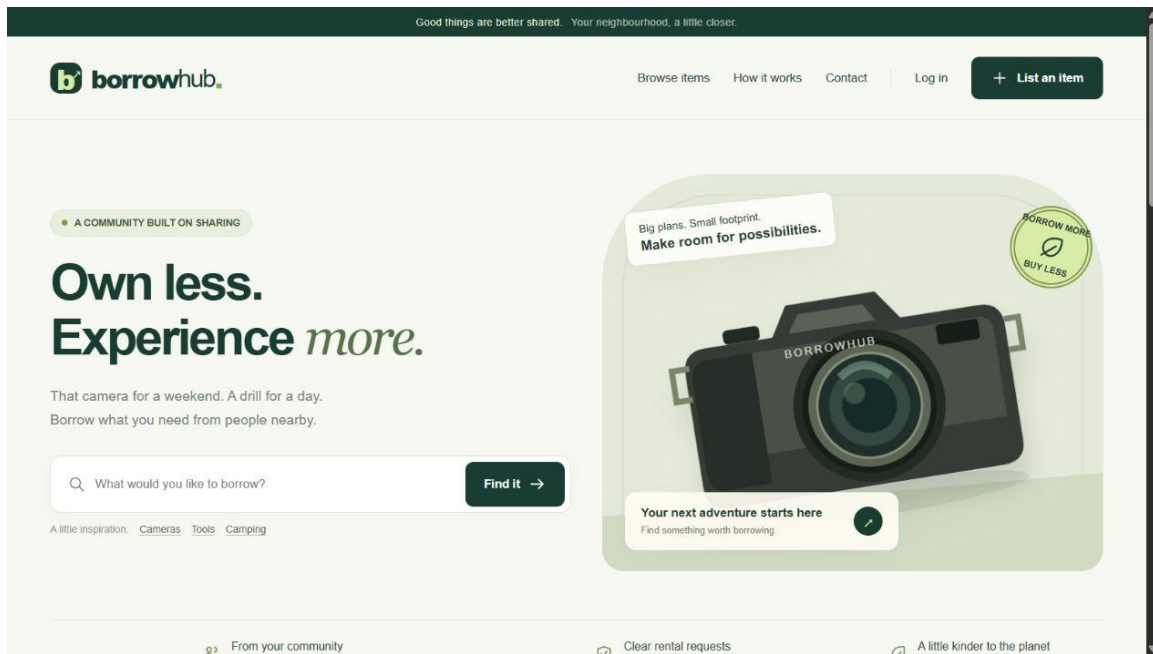


Figure 4. Homepage with the search entry point and community-rental identity.

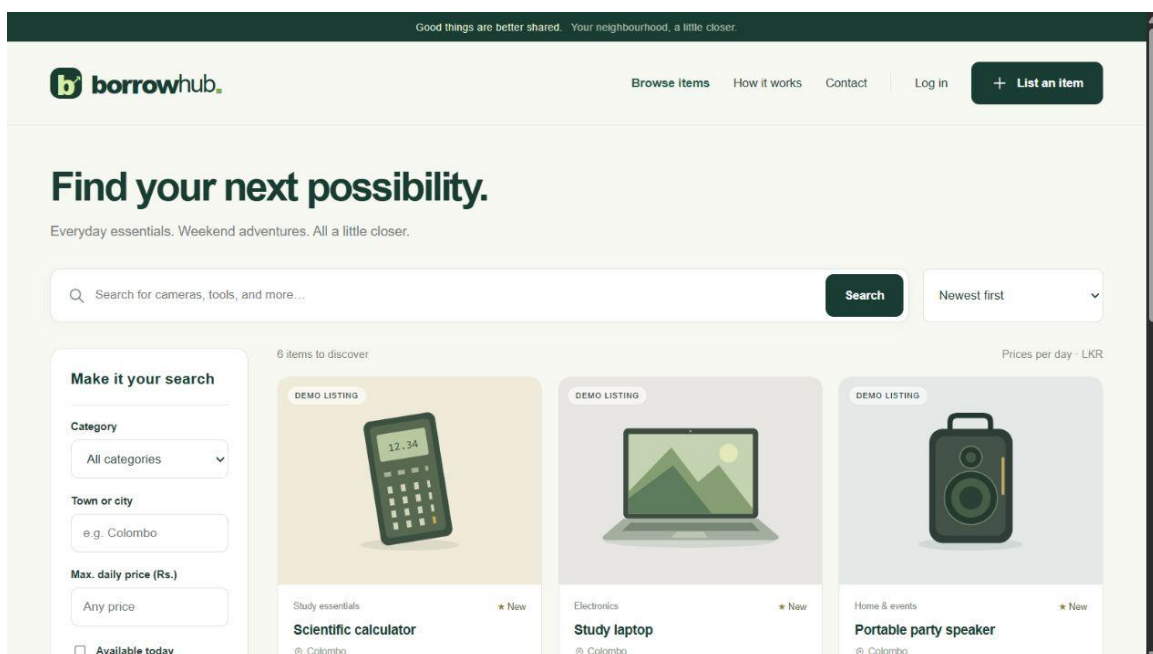
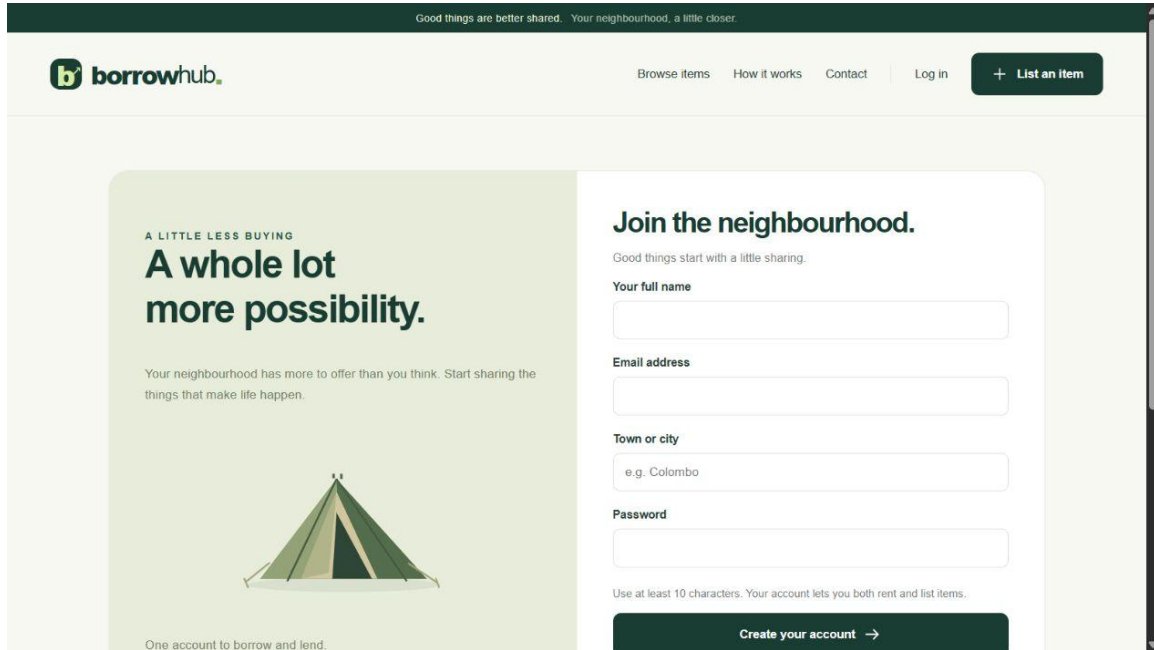


Figure 5. Browse view with category, location, price and availability filters.

## 4.2 Registration and authentication

Registration stores the member's name, email, town and password hash; an ordinary account can both borrow and lend. Login verifies the password and account status, then rotates the session ID. Protected views redirect visitors to login and can return them to the requested item afterward. Invalid input and authentication failures appear next to the form.



Good things are better shared. Your neighbourhood, a little closer.

**b borrowhub.** Browse items How it works Contact Log in [+ List an item](#)

A LITTLE LESS BUYING  
**A whole lot more possibility.**

Your neighbourhood has more to offer than you think. Start sharing the things that make life happen.



One account to borrow and lend.

**Join the neighbourhood.**

Good things start with a little sharing.

Your full name

Email address

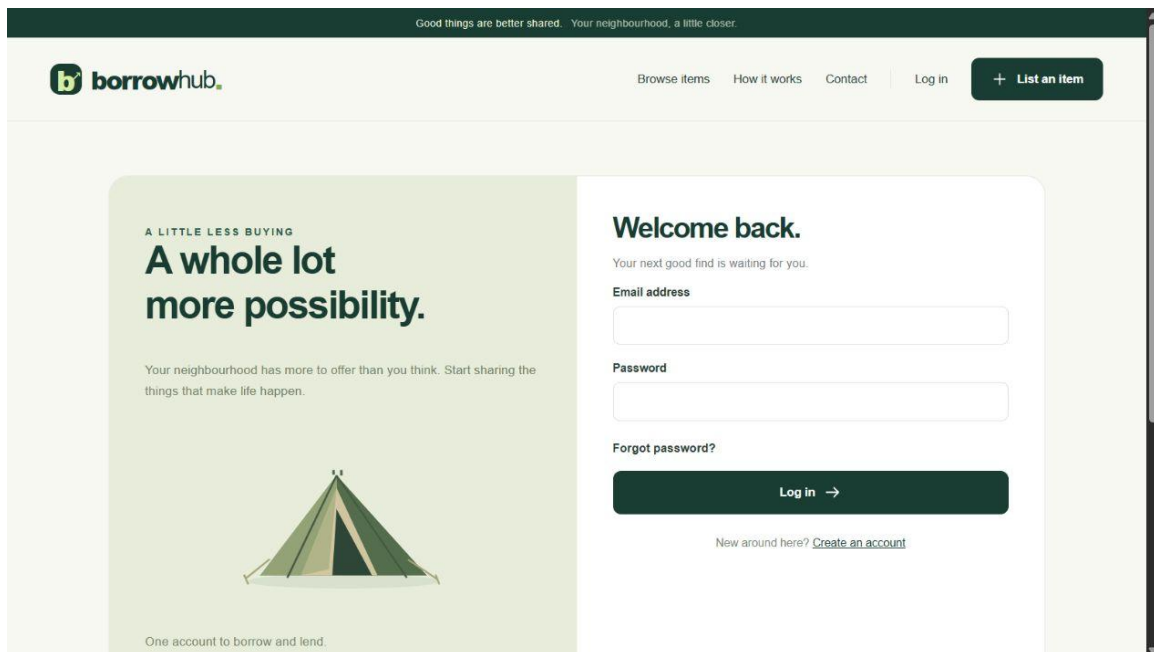
Town or city

Password

Use at least 10 characters. Your account lets you both rent and list items.

[Create your account →](#)

Figure 6. Registration form for a community member.



Good things are better shared. Your neighbourhood, a little closer.

**b borrowhub.** Browse items How it works Contact Log in [+ List an item](#)

A LITTLE LESS BUYING  
**A whole lot more possibility.**

Your neighbourhood has more to offer than you think. Start sharing the things that make life happen.



One account to borrow and lend.

**Welcome back.**

Your next good find is waiting for you.

Email address

Password

[Forgot password?](#)

[Log in →](#)

New around here? [Create an account](#)

Figure 7. Login form and administrator-assisted recovery entry point.

## 4.3 Item details and rental requests

Item details combine the photo or illustration, description, category, condition, owner, location, daily price and deposit. The browser previews the cost as dates change. A signed-in borrower submits a request; the server validates dates, ownership, item availability and existing reservations before calculating and saving the agreement.

**DEMO LISTING**

to content

**Canon EOS 2000D camera**

Colombo Condition: Good Available to request

Shared by Nimal Perera  
Member since 24 Sept 2026

**Rs. 1,800 / day**

Pickup date: 09/25/2026 Return date: 09/25/2026

**A note for the owner (optional)**  
Tell them a little about your plans.

|                      |                  |
|----------------------|------------------|
| Rs. 1,800 × 1 day    | Rs. 1,800        |
| Refundable deposit   | Rs. 5,000        |
| <b>Due at pickup</b> | <b>Rs. 6,800</b> |

Both pickup and return days are charged. Up to 90 days. Final approval comes from the owner.

**Send rental request →**

**Good to know**  
Demonstration listing for the BorrowHub coursework project. Clean, well cared for and ready for your next project. Arrange collection with the owner through your rental messages. Please return all accessories in the same condition.

Payments and the refundable deposit are arranged directly with the owner at pickup. BorrowHub does not collect payments.

**Community reviews ★ 5.0**

**Amaya Silva** ★★★★★  
The camera was in good condition and pickup was straightforward.  
25 Sept 2026

**borrowhub.** Explore Your BorrowHub ROOTED IN COMMUNITY  
The camera was in good condition and pickup was straightforward.  
25 Sept 2026

Figure 8. Item details with booking controls, cost summary and a demonstration review.

### Price calculation

Rental days = (return date - pickup date) + 1  
 Rental total = agreed daily rate × rental days  
 Due at pickup = rental total + refundable deposit

For a camera priced at Rs. 1,800 per day, a two-day rental totals Rs. 3,600. With a Rs. 5,000 deposit, the displayed amount due at pickup is Rs. 8,600. Both boundary dates are chargeable. This is an agreement summary, not an online payment transaction.

## 4.4 Listing and image management

Members publish items with a title, description, category, condition, town, rate and deposit. Existing listings can be edited or paused. Image uploads accept JPG, PNG and WebP up to 5 MB and 8000 × 8000 pixels, subject to PHP upload limits. MIME and image checks run on the server, and accepted files receive random names.

The screenshot shows the 'What are you sharing?' form on the BorrowHub website. The form is divided into several sections:

- Title:** A text input field with the placeholder 'e.g. Canon EOS camera with kit lens'.
- Category:** A dropdown menu with 'Select a category' as the current selection.
- Town or city:** A text input field with 'Colombo' entered.
- Tell us about it:** A large text area for description, with placeholder text 'Condition, included accessories, pickup arrangements, and anything borrowers should know...'.
- Daily rental price (Rs.):** A text input field.
- Deposit (Rs.):** A text input field with '0' entered.
- Condition:** A dropdown menu with 'Good' selected.
- Availability:** A dropdown menu with 'Open to requests' selected.
- Item photo (optional):** A section with a 'Choose File' button and the text 'No file chosen'.

Below the form, there is a 'Publish listing →' button and a 'Cancel' button. A dark green notification bubble with the text 'Welcome back.' is positioned over the rental price and deposit fields. At the bottom of the page, there is a footer with three columns: 'Explore' (with a link to 'Browse items'), 'Your BorrowHub' (with a link to 'My rentals'), and 'ROOTED IN COMMUNITY' (with a link to 'A small idea with a lighter footprint.').

Figure 9. Listing form, including rental pricing, availability and photo upload.

Only the listing owner can edit or remove it. Removal uses a soft-deletion timestamp so rental history is retained. The server blocks removal while confirmed rentals remain. Listing moderation and owner availability are separate controls: an administrator can hide a listing even if the owner has marked it available.

## 4.5 Rental dashboard and returns

The member dashboard separates “I’m borrowing,” “I’m lending” and “My listings.” Cards show the other participant, dates, agreed amount, deposit, status and permitted next actions. Summary cards show listed items, confirmed/active rentals, requests awaiting review and completed rental value. Due and overdue messages are calculated when the dashboard is loaded or refreshed.

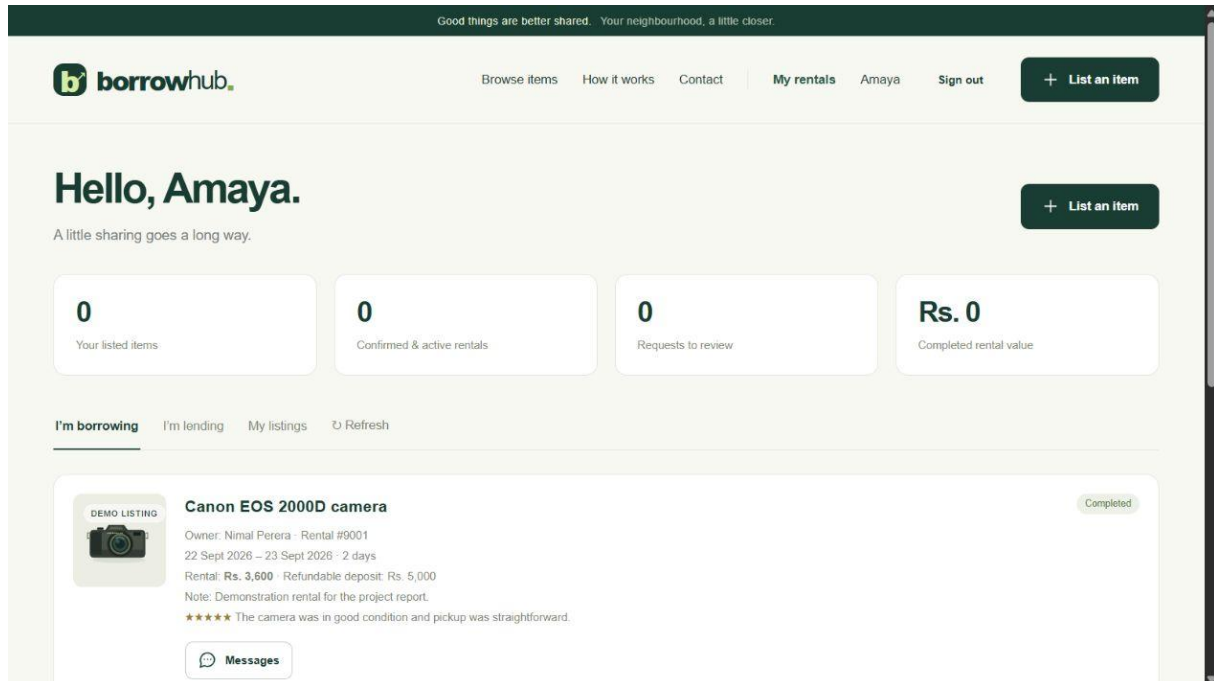


Figure 10. Borrower dashboard with a completed demonstration rental and rental history.

| Action                      | Who can perform it                          | Result                                                         |
|-----------------------------|---------------------------------------------|----------------------------------------------------------------|
| Send request                | Borrower                                    | Creates a pending rental.                                      |
| Accept / decline            | Item owner                                  | Reserves valid dates or rejects the request.                   |
| Confirm pickup              | Item owner                                  | Changes an accepted rental to active during the rental period. |
| Request return confirmation | Borrower                                    | Changes active to return_requested.                            |
| Confirm returned            | Item owner                                  | Completes the rental after return.                             |
| Cancel before pickup        | Borrower: pending/accepted; owner: accepted | Changes the rental to cancelled.                               |

Accepting one request declines other pending requests that overlap the same dates. Completed-rental value is an agreed price total; the application does not verify that money has changed hands.

## 4.6 Private messages and reviews

Messages are attached to a rental rather than a public listing. The server allows only its borrower and owner to read or send them. After completion, the borrower can submit one rating from 1 to 5 and an optional comment. Reviews appear on the item page. The examples below use fictional demonstration records.

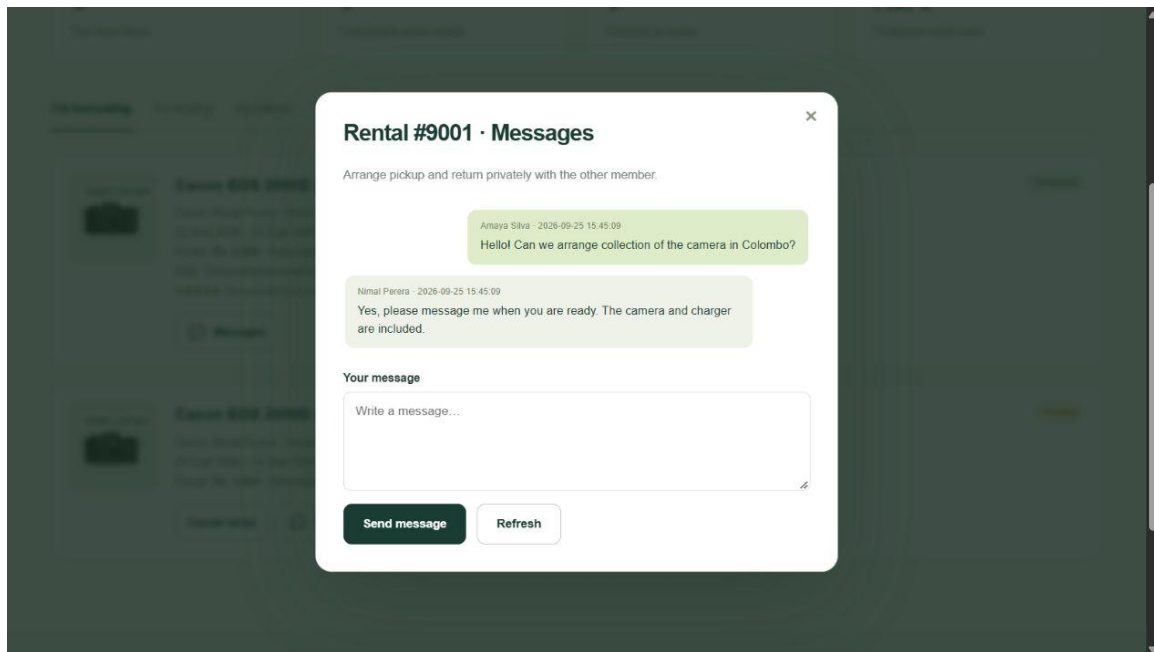


Figure 11. Rental-specific conversation between two demo members.

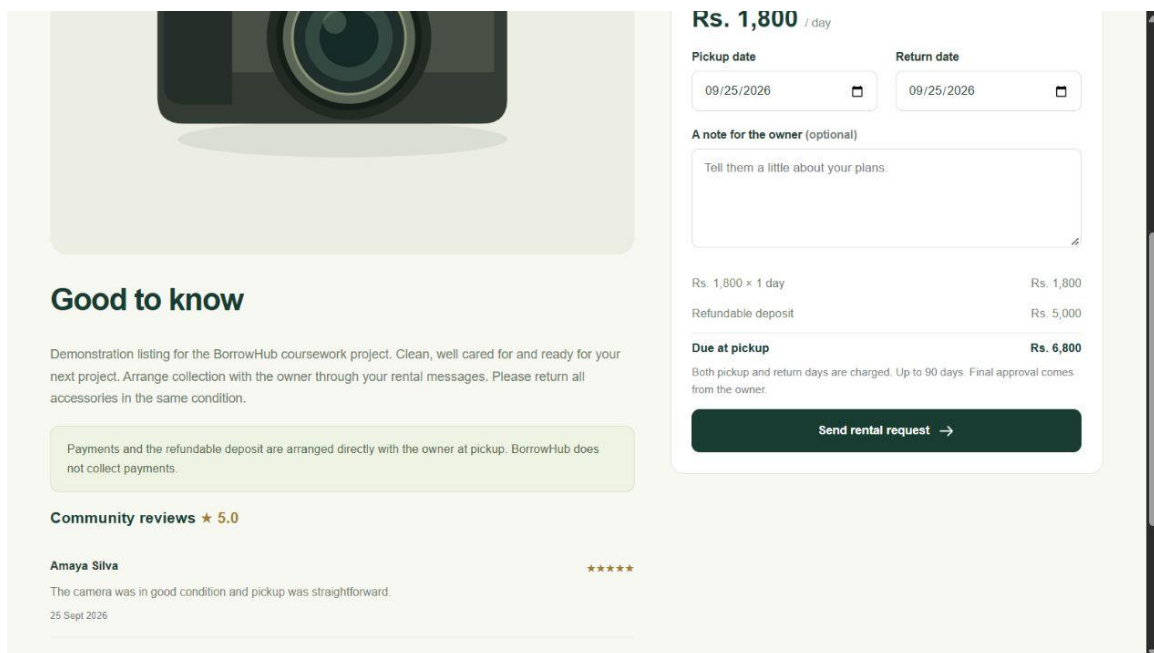


Figure 12. Completed-rental review displayed on the item details page.

## 4.7 Profiles and customer support

Members can update their name, town and optional phone number, or change a password after supplying the current password. Public listings do not expose their email or phone. The contact form stores the sender, subject and message for administrator review; it does not claim to send an email.

The screenshot shows the BorrowHub website interface. At the top, a dark green header contains the slogan "Good things are better shared. Your neighbourhood, a little closer." and the BorrowHub logo. A navigation bar includes links for "Browse items", "How it works", "Contact", "My rentals", "Amaya", "Sign out", and a "+ List an item" button. The main heading is "Your corner of the community." with a subtext "Keep your details up to date." Below this, there are two side-by-side forms. The left form, titled "Profile details", has input fields for "Full name" (containing "Amaya Silva"), "Town or city" (containing "Kandy"), and "Phone number (optional)". A note states "Your email: amaya@example.test. Phone details stay private and are not included in public listings." and a "Save profile" button is at the bottom. The right form, titled "Change password", has input fields for "Current password" and "New password". A note says "At least 10 characters. Other active sessions will be signed out." and an "Update password" button is at the bottom.

Figure 13. Profile update and password-change forms.

The screenshot shows the BorrowHub website interface. At the top, a dark green header contains the slogan "Good things are better shared. Your neighbourhood, a little closer." and the BorrowHub logo. A navigation bar includes links for "Browse items", "How it works", "Contact", "My rentals", "Amaya", "Sign out", and a "+ List an item" button. The main heading is "Let's talk." with a subtext "A question, an idea, or a little help with a rental?". Below this, there is a contact form on the left and a support guidance section on the right. The contact form has input fields for "Your name" (containing "Amaya Silva"), "Email address" (containing "amaya@example.test"), "Subject" (containing "How can we help?"), and "Your message" (containing "Tell us a little more..."). A "Send message" button with a right arrow is at the bottom. The support guidance section, titled "HERE FOR THE NEIGHBOURHOOD", has the heading "Good connections start with a hello." and a note "Your message goes directly to the BorrowHub administration dashboard." Below this, there are two sections: "About a specific rental?" with the text "Use Messages in My rentals to speak privately with the owner or borrower." and "Need account recovery?" with the text "Contact the project administrator to verify your identity and request a one-time password reset token."

Figure 14. Contact form and guidance for rental or account support.

## 4.8 Administration

The administrator dashboard provides totals and management views for items, users, rentals, contacts and categories. Administrators can hide listings, suspend eligible member accounts, resolve contact messages and add or disable categories. Server-side role checks protect these actions; normal members cannot obtain admin records by calling the endpoint directly.

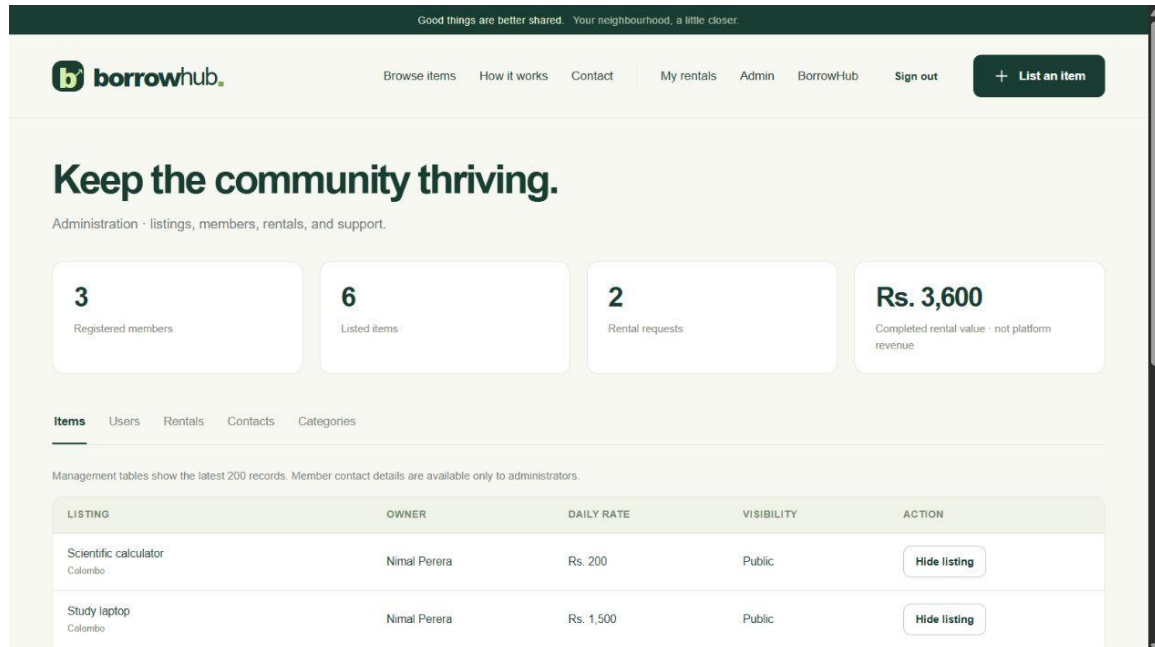


Figure 15. Administrator summary and listing-moderation table.

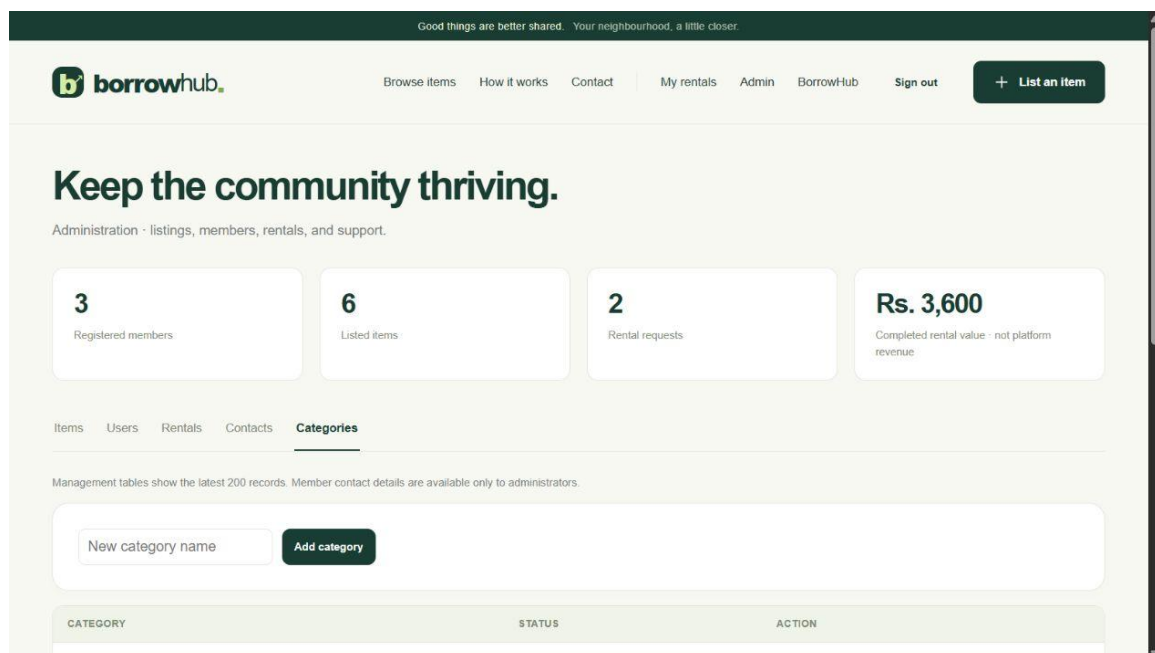


Figure 16. Category management with add, enable and disable controls.

## 4.9 Security, validation and testing

| Concern              | Implemented control                                                                         |
|----------------------|---------------------------------------------------------------------------------------------|
| Password storage     | password_hash / password_verify; no plaintext password column [7].                          |
| Session security     | Session-ID rotation, HTTP-only SameSite cookies and session-version invalidation.           |
| Cross-site requests  | Session-bound CSRF token required on every POST mutation.                                   |
| SQL injection        | Prepared PDO statements; fixed/validated identifiers and sort options.                      |
| Stored markup        | User text is escaped before it is inserted into client-rendered HTML.                       |
| Unauthorized actions | Server checks the account role, listing owner and rental participants.                      |
| Unsafe uploads       | MIME and image validation, size/dimension limits and random stored filenames.               |
| Abuse                | Per-IP throttling for selected authentication, booking, message and contact operations.     |
| Private files        | Apache access rules and the development router block configuration, schema and credentials. |

### Recorded verification results

The implementation record reports 34 successful real HTTP/MySQL integration checks on 24 September 2026, alongside PHP and JavaScript syntax checks [3]. These cover access control, booking conflicts, price snapshots, pickup and return, messages, reviews, uploads, moderation, recovery and history retention. The suite creates and removes a randomly named test database; it does not use the application database for its test records.

| Verification area | Evidence                                                                                       |
|-------------------|------------------------------------------------------------------------------------------------|
| Workflow          | A rental was requested, approved, picked up, returned, completed and reviewed.                 |
| Permissions       | Anonymous/member/admin boundaries and private-message access were checked.                     |
| Data integrity    | Duplicate reviews, conflicting dates and listing removal with confirmed rentals were rejected. |
| Frontend          | Search, totals, login redirects, booking submission and admin pages were browser-tested.       |
| Responsive layout | Home and member dashboard were checked at a 390px viewport without horizontal overflow.        |

The checks establish local functional behaviour; they are not a load test or an independent security audit. This version uses manual message refresh and in-app reminders. Administrator tables show the most recent 200 records, and historical/replaced uploads are not automatically purged.

## 5. Challenges and solutions

The following distinguishes implemented design safeguards from issues observed during development and report preparation. It does not assume that every safeguard originated in a separately recorded bug.

| Challenge                                                      | Solution / handling                                                                             | Outcome                                                                                                                 |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Preventing conflicting rentals                                 | Lock the item row inside an InnoDB transaction; recheck the overlap before approval.            | One confirmed agreement can occupy a given date range for the item. Competing pending requests are declined.            |
| Keeping agreement prices stable                                | Store days, daily_rate, total and deposit on the rental when requested.                         | Editing a listing does not change an existing rental's agreed amount.                                                   |
| Preserving history when listings are removed                   | Soft-delete listings; reject removal while confirmed rentals exist.                             | Past rentals remain accessible and related records retain their foreign keys.                                           |
| Protecting photos and private data                             | Validate real image content, store random filenames, escape displayed text and check ownership. | Disguised executable uploads and unauthorized operations were rejected in tests.                                        |
| PHP session storage permission failure during isolated testing | Run the authorised test server with access to PHP's configured session directory.               | The test suite could retain cookies and CSRF tokens and complete authentication checks.                                 |
| Search shortcut failed to find the demo camera                 | Make the demo listing title explicitly include "camera."                                        | The homepage camera shortcut returned the expected item.                                                                |
| First-load CSRF mismatch during screenshot capture             | Reload the page to obtain a matching session token, then sign in again.                         | Login succeeded. This is a recovery step; the observation is retained as a session-initialisation follow-up.            |
| IIS serving localhost during report capture                    | Use the implemented PHP development router on a separate port for demo screenshots.             | Screenshots were captured without changing IIS settings. XAMPP users must verify which server owns their selected port. |

### Future improvements

Possible extensions include email-based recovery and reminders, a real payment integration, improved admin pagination, automated unused-image cleanup, location-distance search and more extensive concurrent-load tests. The first-load session/token behaviour should also be investigated before a public deployment. These are future work rather than features claimed as complete.

## 6. Individual contribution

The group confirmed that all work was performed as a team. Accordingly, both members are recorded as joint contributors throughout the project. The report does not invent separate frontend/backend ownership, hours worked or percentage allocations.

| Member          | Index number | Contribution                                                                                                                       |
|-----------------|--------------|------------------------------------------------------------------------------------------------------------------------------------|
| K. M. Aashik    | ITT/2024/001 | Joint participation in requirements, interface and database design, implementation, integration, testing, setup and documentation. |
| M. R. M. Rislam | ITT/2024/090 | Joint participation in requirements, interface and database design, implementation, integration, testing, setup and documentation. |

### Shared contribution by activity

| Activity                  | Aashik | Rislam | Shared output                                                      |
|---------------------------|--------|--------|--------------------------------------------------------------------|
| Proposal and scope        | Joint  | Joint  | Rental-platform purpose, audience and planned features.            |
| System design             | Joint  | Joint  | Application structure, entities and rental workflow.               |
| Frontend work             | Joint  | Joint  | Responsive pages, forms, navigation and visual presentation.       |
| Backend and database work | Joint  | Joint  | Accounts, listings, rentals, messages, reviews and administration. |
| Integration and testing   | Joint  | Joint  | End-to-end checking, validation and corrections.                   |
| Setup and documentation   | Joint  | Joint  | XAMPP instructions, source organisation and project report.        |

### Team statement

“We did everything as a team.” The same shared responsibility therefore appears for both members in the breakdown. The completed application and documentation are presented as one collaborative group project.

## 7. Setup instructions

### 7.1 Requirements and folder placement

Use a Windows XAMPP installation with PHP 8.1 or later and MySQL 5.7+ or MariaDB 10.3+. The development environment used PHP 8.2.12 and MariaDB 10.4.32. Enable PDO MySQL, mbstring, fileinfo and PHP sessions. Node.js is needed only for the optional integration tests, not for running the website.

1. Copy the complete BorrowHub folder to C:\xampp\htdocs\borrowhub. Keep its own assets, database and uploads subfolders together.
2. Open XAMPP Control Panel and start Apache and MySQL. Confirm they are running before opening the site. The project requires an HTTP server that executes PHP, not a file:/// URL or VS Code Live Server [8].

### 7.2 Import the database using phpMyAdmin

3. Open <http://localhost/phpmyadmin/> in the browser. If Apache uses a different port, include that port in the URL.
4. Choose New, create a database named borrowhub and select utf8mb4\_unicode\_ci as the collation.
5. Select borrowhub in the database list. Open Import, choose C:\xampp\htdocs\borrowhub\database\schema.sql, keep SQL as the format, and select Import/Go [9].
6. Confirm that the eight tables exist: users, categories, items, rentals, reviews, messages, contacts and password\_resets.
7. The schema file creates tables only; it does not select the database or create login accounts. Complete the installer below to add the starting categories and administrator.

### 7.3 Configure and initialise

| Setting in config.php | Typical local value                              |
|-----------------------|--------------------------------------------------|
| host / port           | 127.0.0.1 / 3306                                 |
| database              | borrowhub                                        |
| username / password   | root / empty on a fresh local XAMPP installation |
| timezone              | Asia/Colombo                                     |

Use the actual MySQL credentials if your XAMPP installation differs. Open PowerShell in the project folder and run:

```
cd C:\xampp\htdocs\borrowhub
powershell -ExecutionPolicy Bypass -File .\install.ps1
```

Enter the administrator email and a password of 10–72 bytes when prompted. Choose whether to add demo data and supply a separate demo password. The installer can also create the database and schema itself, so the manual import above is an alternative entry point, not a prerequisite. Re-running setup preserves existing accounts and records.

## 7.4 Run and verify the application

8. Visit <http://localhost/borrowhub/> after Apache and MySQL are running. If Apache is configured on port 8080, use <http://localhost:8080/borrowhub/> instead.
9. Sign in using the administrator credentials chosen during setup. If this machine already has a generated `.local-credentials.txt` file, read it locally for the installed credentials. Do not place those passwords in a public report.
10. If demo data was selected, use `nimal@example.test` as the owner and `amaya@example.test` as the borrower, with the demo password chosen at installation. Alternatively, register new member accounts.
11. Browse an item, request valid dates as the borrower, and accept it as the owner. Confirm pickup, request return confirmation, complete the return and submit a review.
12. Submit a contact message and verify that it appears in Admin → Contacts. Check listing and category controls with the administrator account.

### Optional development-server command

If Apache's usual port is occupied, MySQL can still be used with the supplied PHP router on a free local port:

```
cd C:\xampp\htdocs\borrowhub
& C:\xampp\php\php.exe -d upload_max_filesize=5M `
-d post_max_size=6M -S 127.0.0.1:8088 router.php
```

For that command, open <http://127.0.0.1:8088/> (without `/borrowhub/`). The router protects private project files. Keep the terminal running while using the site.

### Troubleshooting

| Symptom                             | Check / solution                                                                                                                                     |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| IIS 404 or Apache cannot start      | Another service may own port 80. Use an available Apache port, or the temporary PHP-server option; verify the correct URL.                           |
| Database connection fails           | Start MySQL; check host, port, database and credentials in <code>config.php</code> .                                                                 |
| Categories or admin account missing | Run <code>install.ps1</code> or <code>setup.php</code> with the appropriate administrator environment variables.                                     |
| Session changed / CSRF error        | Reload the page and retry. Ensure cookies are allowed and PHP's session directory is writable.                                                       |
| Large photo rejected                | Check PHP <code>upload_max_filesize</code> and <code>post_max_size</code> as well as the app's 5 MB image limit.                                     |
| Forgotten password                  | After identity verification, the administrator runs <code>reset-password.php</code> with the member email and supplies the one-time token privately. |

## 8. References

Project-specific statements are drawn primarily from the proposal, code, schema and verification record. Official documentation below supports the technologies and setup procedures. Online references were accessed on 25 September 2026. No external photographs are used in the application screenshots; the illustrations are included in assets/art.js.

- [1] K. M. Aashik and M. R. M. Rislam. Borrow Hub — Smart Community Rental Platform, ICT1209 Web Technologies Project Proposal. Supplied eight-page document, “Project Proposal..pdf.”
- [2] BorrowHub source files. api.php, lib.php, config.php, assets/app.js, assets/style.css, database/schema.sql and README.md. Local implementation inspected September 2026.
- [3] BorrowHub verification records. VERIFICATION.md and tests/integration.cjs; 34 integration checks recorded as passed on 24 September 2026.
- [4] MDN Web Docs. “Fetch API.” [https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API)
- [5] PHP Documentation. “PHP Data Objects (PDO).” <https://www.php.net/manual/en/book.pdo.php>
- [6] Oracle. MySQL 8.4 Reference Manual, “Locking Reads.” <https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html>
- [7] PHP Documentation. “password\_hash.” <https://www.php.net/manual/en/function.password-hash.php>
- [8] Apache Friends. “XAMPP FAQs for Windows.” [https://www.apachefriends.org/faq\\_windows.html](https://www.apachefriends.org/faq_windows.html)
- [9] phpMyAdmin Documentation. “Import and export.” [https://docs.phpmyadmin.net/en/latest/import\\_export.html](https://docs.phpmyadmin.net/en/latest/import_export.html)
- [10] MDN Web Docs. “Using media queries.” [https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media\\_queries/Using](https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using)

### Implementation note

Responsive styling uses custom CSS media queries [10]. Bootstrap was mentioned in the original proposal but is not a runtime dependency of the delivered project. The cited MySQL locking documentation explains the design principle; the local verification environment used MariaDB 10.4.32.

### Conclusion

BorrowHub provides a working local platform for community item rentals, from discovery and request approval through pickup, return and review. The database, permission checks and recorded tests support the central workflow, while the documented limits keep the report aligned with what the application actually implements.